CASE STUDY REPORT

SMART FARMING & CROP INTELLIGENCE MANAGEMENT SYSTEM

Problem Statement No: 121

Submitted By:

Yashika Saraf

Roll No:

150096725035

University:

ITM Skills University

Subject:

Data Structures and Algorithms with C++

1. Project Title

Smart Farming & Crop Intelligence Management System

2. Problem Statement

Agricultural organizations require a platform to monitor crop health, manage farm resources, predict yields, optimize irrigation schedules, and support data-driven farming decisions to improve agricultural productivity.

Traditional farming methods often rely on manual observation and experience-based decisions, which can lead to inefficient use of water, fertilizers, and other resources. Farmers often face challenges in monitoring crop conditions across large farms, managing irrigation efficiently, and responding quickly to environmental changes.

The Smart Farming & Crop Intelligence Management System aims to solve these challenges by providing a centralized platform for crop monitoring, resource management, irrigation planning, and farm analysis using various Data Structures and Algorithms.

The system helps improve agricultural productivity while reducing operational costs and resource wastage.

3. Objectives

The main objectives of the project are:

• Monitor crop health and growth status.

• Manage irrigation schedules efficiently.

• Track farm resources such as water and fertilizer.

• Store and retrieve crop information efficiently.

• Process farm alerts and notifications in real time.

• Optimize irrigation routes using graph algorithms.

• Support yield prediction and crop prioritization.

• Improve farm productivity through data-driven decision making.

• Reduce water wastage and operational expenses.

4. Project Description

The Smart Farming & Crop Intelligence Management System is a C++ based application designed to assist farmers and agricultural organizations in managing farming activities more efficiently.

The system provides features for crop management, irrigation scheduling, alert processing, resource allocation, and route optimization. It utilizes various Data Structures and Algorithms to ensure efficient storage, retrieval, and processing of agricultural data.

The application allows users to:

• Add crop records.

• Search crop information.

• Update crop details.

• Manage irrigation plans.

• Process sensor alerts.

• Analyze farm networks.

• Find shortest irrigation routes.

• Monitor farm productivity.

The project demonstrates the practical implementation of Data Structures and Algorithms in the Agriculture and AgriTech industry.

---

5. Technologies and DSA Concepts Used

Programming Language

• C++

Data Structures Used

Stack

Used to manage irrigation plans and support undo operations.

Queue

Used for processing sensor alerts and notifications.

Binary Search Tree (BST)

Used for storing crop records efficiently.

AVL Tree

Used for maintaining balanced crop databases and improving search performance.

Hashing

Used for fast crop identification and retrieval.

Algorithms Used

Sorting Algorithms

Used to prioritize crops based on health score and yield potential.

Searching Algorithms

Used for efficient crop record retrieval.

Breadth First Search (BFS)

Used to analyze farm connectivity level by level.

Depth First Search (DFS)

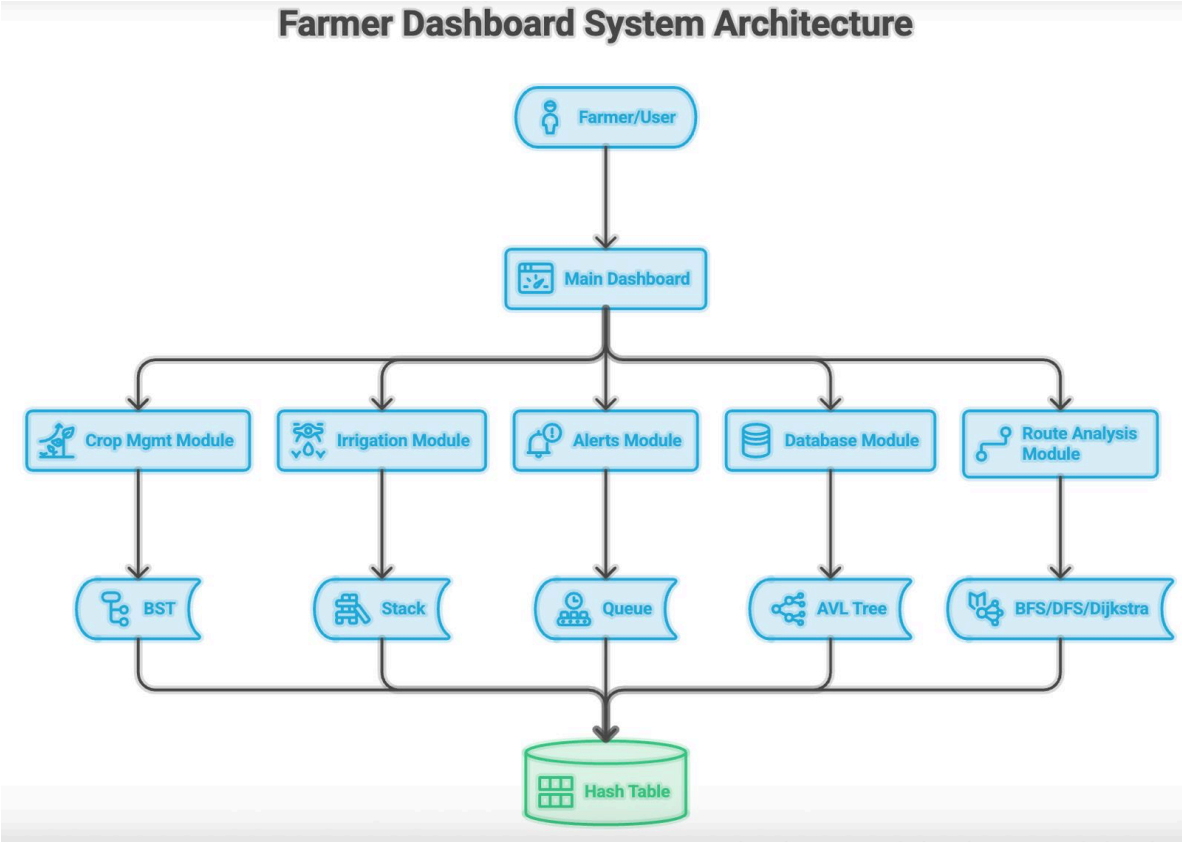
Used to explore complete farm networks.

Dijkstra Algorithm

Used to determine shortest irrigation routes.

---

6. System Architecture



7. Algorithm Description

Sorting Algorithm

Purpose:

To prioritize crops according to health score and expected yield.

Working:

1. Collect crop records.
2. Compare health scores.
3. Arrange crops from highest to lowest priority.
4. Display prioritized crop list.

Example:

Before Sorting

Rice = 80

Wheat = 95

Corn = 70

After Sorting

Wheat = 95

Rice = 80

Corn = 70

Searching Algorithm

Purpose:

To retrieve crop records efficiently.

Working:

1. Enter Crop ID.
2. Search database.

3. Display crop information.

Time Complexity:

Linear Search = O(n)

Binary Search = O(log n)

Stack

Purpose:

Manage irrigation plans.

Operations:

Push – Add irrigation plan.

Pop – Remove latest irrigation plan.

Time Complexity:

Push = O(1)

Pop = O(1)

Queue

Purpose:

Process farm alerts and notifications.

Working:

Alerts are processed in FIFO order.

Example:

Alert 1

Alert 2

Alert 3

Processing Order:

Alert 1 → Alert 2 → Alert 3

Time Complexity:

Enqueue = O(1)

Dequeue = O(1)

Binary Search Tree (BST)

Purpose:

Store crop records.

Operations:

• Insert

• Search

• Delete

Average Time Complexity:

O(log n)

Worst Case:

O(n)

AVL Tree

Purpose:

Maintain balanced crop database.

Advantages:

• Faster searching.

• Balanced tree structure.

• Improved performance for large datasets.

Time Complexity:

Insert = O(log n)

Search = O(log n)

Delete = O(log n)

Breadth First Search (BFS)

Purpose:

Analyze farm fields level by level.

Time Complexity:

O(V + E)

Applications:

• Farm connectivity analysis.

• Resource distribution planning.

Depth First Search (DFS)

Purpose:

Explore complete farm network.

Time Complexity:

O(V + E)

Applications:

• Route exploration.

• Farm network analysis.

Dijkstra Algorithm

Purpose:

Find the shortest irrigation route.

Example:

A → B = 5

A → C = 2

C → D = 3

Shortest Path:

A → C → D

Total Cost = 5

Time Complexity:

O((V + E) log V)

Hashing

Purpose:

Enable rapid crop identification.

Advantages:

• Fast searching.

• Efficient storage.

Average Time Complexity:

O(1)

---

8. Complexity Analysis

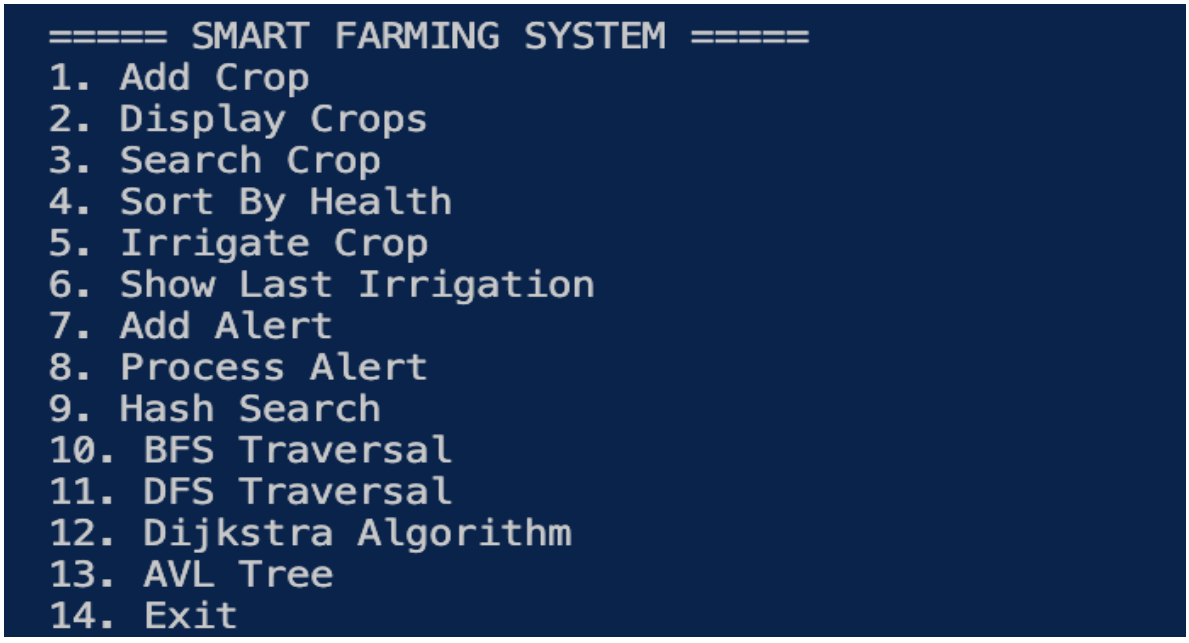
Operation	Data Structure/Algorithm	Complexity
Crop Search	Hashing	O(1)
Crop Insert	BST	O(log n)
Crop Search	BST	O(log n)
AVL Insert	AVL Tree	O(log n)
AVL Search	AVL Tree	O(log n)
Push Plan	Stack	O(1)
Pop Plan	Stack	O(1)

Add Alert	Queue	O(1)
Process Alert	Queue	O(1)
BFS Traversal	Graph	O(V+E)
DFS Traversal	Graph	O(V+E)
Dijkstra Route	Graph	O((V+E)logV)

9. Screenshots of Execution

Screenshot

– Main Dashboard



Screenshot 1 – Add Crops

```
Enter Choice: 1

Enter Crop ID: 101
Enter Crop Name: Wheat
Enter Health Score: 90
Crop Added Successfully!
```

Screenshot 2 – Display Crops

```
Enter Choice: 2

Crop Records
ID: 101 Name: Wheat Health: 90
ID: 102 Name: Rice Health: 75
ID: 103 Name: Corn Health: 89
```

Screenshot 3 – Search Crop

```
Enter Choice: 3
Enter Crop ID: 101
Crop Found: Wheat Health: 90
```

Screenshot 4 – Sort by health

```
Enter Choice: 4
Sorted By Health Score
```

Screenshot 5 – Irrigation Crop

```
Enter Choice: 5
Enter Crop Name: Rice
Rice Irrigated Successfully
```

Screenshot 6 – Showing last Irrigation

```
Enter Choice: 6
Last Irrigated Crop: Rice
```

Screenshot 7– Add Alert

```
Enter Choice: 7
Enter Alert: Low soil moisture
Alert Added
```

Screenshot 8 – Process Alert

```
Enter Choice: 8
Processing Alert: Low soil moisture
```

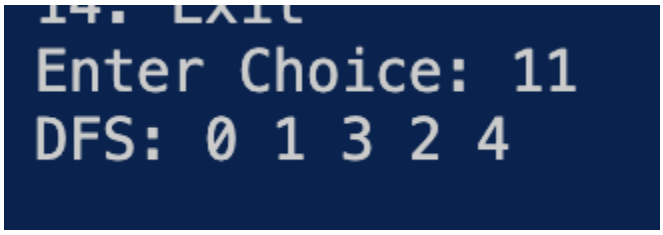
Screenshot 9– Hash Search

```
Enter Choice: 9
Enter Crop ID: 102
Crop Found: Rice
```

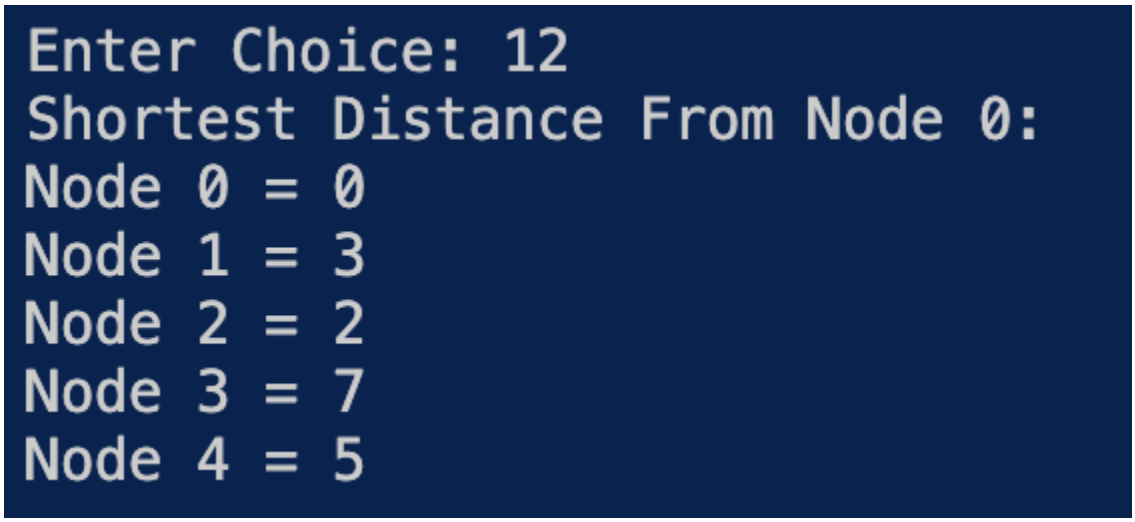
Screenshot 10 – BFS Traversal

```
Enter Choice: 10
BFS: 0 1 2 3 4
```

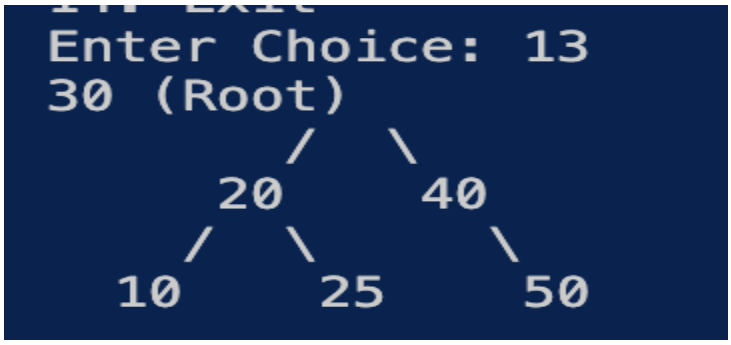
Screenshot 11– DFS Traversal



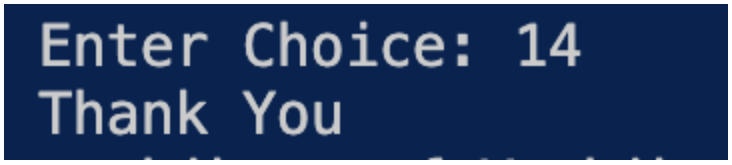
Screenshot 12 - Dijkstra Algorithm



Screenshot 13- AVL Tree



Screenshot 14 - Exit



10. Results and Findings

The developed system successfully demonstrated the implementation of multiple Data Structures and Algorithms in the agriculture domain.

Results obtained:

• Efficient storage of crop records.

• Faster retrieval of crop information using Hashing.

• Effective irrigation management through Stack operations.

• Real-time alert processing using Queue.

• Improved route optimization using Dijkstra Algorithm.

• Better farm network analysis through BFS and DFS.

Findings:

• Hashing provides the fastest search performance.

• AVL Trees perform better than BSTs for large datasets.

• Queue is highly effective for alert management.

• Dijkstra Algorithm significantly reduces irrigation route cost.

• Smart resource allocation improves productivity and efficiency.

11. Conclusion

The Smart Farming & Crop Intelligence Management System successfully integrates Data Structures and Algorithms to solve real-world agricultural challenges.

The project demonstrates practical implementation of Sorting, Searching, Stack, Queue, BST, AVL Trees, Hashing, BFS, DFS, and Dijkstra Algorithms.

The system improves crop management, irrigation planning, resource allocation, and route optimization while supporting data-driven farming decisions.

Overall, the project contributes to modern smart agriculture by improving productivity, reducing resource wastage, and enhancing operational efficiency.

---

12. GitHub Repository

https://github.com/yashikasaraff7-hub/DSA-PROJECT